Here's a **complete explanation** of **Attributes and Variables** in Python OOP:

🎯 FIRST — WHAT IS THE DIFFERENCE?

In Python OOP:

```
Variable   → General term for storing data
Attribute  → Variable that belongs to
             a CLASS or OBJECT
```

Simple Rule:

→ Outside class = called VARIABLE

→ Inside class = called ATTRIBUTE

```
x = 10          # this is a VARIABLE

class MyClass:
    x = 10        # this is an ATTRIBUTE
    self.x = 10     # this is also ATTRIBUTE
```

🗂️ COMPLETE MAP OF ATTRIBUTES

```
ALL ATTRIBUTES IN PYTHON OOP
|
├── 1. Class Attribute
|      → Belongs to CLASS
|      → Shared by ALL objects
|
├── 2. Instance Attribute
|      → Belongs to OBJECT
|      → Unique to each object
|      → Defined in __init__
|
```

```
├── 3. Class Method Attribute
│       → Method belonging to class
│       → Uses @classmethod decorator
│
├── 4. Static Attribute
│       → Independent method
│       → Uses @staticmethod decorator
│
└── 5. Special/Magic Attributes
        → Built-in Python attributes
        → __name__, __doc__, __dict__
```

1️⃣ CLASS ATTRIBUTE

python

```python
class Student:

    # CLASS ATTRIBUTES
    # → Defined directly in class body
    # → Outside any method
    # → Shared by ALL instances
    school   = "ABC School"    # class attribute
    country  = "India"         # class attribute
    count    = 0               # class attribute

    def __init__(self, name, marks):
        self.name  = name      # instance attribute
        self.marks = marks     # instance attribute
        Student.count += 1     # modifying class attribute



# Creating objects
```

```
s1 = Student("Rahul", 90)
s2 = Student("Priya", 85)
s3 = Student("Amit",  78)

# Class attribute → SAME for all objects
print(s1.school)      # ABC School
print(s2.school)      # ABC School
print(s3.school)      # ABC School

# Access via CLASS directly
print(Student.school)  # ABC School
print(Student.count)   # 3

# Instance attribute → DIFFERENT per object
print(s1.name)        # Rahul
print(s2.name)        # Priya
print(s3.name)        # Amit
```

When class attribute is modified:

python

```
class Demo:
    value = 100    # class attribute

d1 = Demo()
d2 = Demo()

# Modify via CLASS → affects ALL objects
Demo.value = 999
print(d1.value)   # 999
print(d2.value)   # 999

# Modify via OBJECT → creates new instance attribute
```

```
# Does NOT change class attribute
d1.value = 500
print(d1.value)   # 500 → now instance attribute
print(d2.value)   # 999 → still class attribute
print(Demo.value) # 999 → class attribute unchanged
```

2️⃣ INSTANCE ATTRIBUTE

python

```
class Employee:

    # Class attribute
    company = "TCS"

    def __init__(self, name, salary, dept):
        # INSTANCE ATTRIBUTES
        # → Defined inside __init__
        # → Using self.attribute_name
        # → Unique to EACH object
        self.name   = name      # instance attribute
        self.salary = salary    # instance attribute
        self.dept   = dept      # instance attribute
        self._id    = id(self)  # instance attribute


# Each object has OWN copy of instance attributes
e1 = Employee("Rahul", 50000, "QA")
e2 = Employee("Priya", 75000, "Dev")
e3 = Employee("Amit",  60000, "HR")

print(e1.name,  e1.salary)   # Rahul 50000
print(e2.name,  e2.salary)   # Priya 75000
```

```
print(e3.name,  e3.salary)   # Amit  60000

# Changing one object does NOT affect others
e1.salary = 55000
print(e1.salary)   # 55000 → changed
print(e2.salary)   # 75000 → unchanged
print(e3.salary)   # 60000 → unchanged
```

Adding instance attribute dynamically:

python

```
class Person:
    def __init__(self, name):
        self.name = name

p = Person("Rahul")
print(p.name)      # Rahul

# Add new attribute DYNAMICALLY
p.age    = 25      # added after instantiation
p.city   = "Bangalore"

print(p.age)       # 25
print(p.city)      # Bangalore

# Other objects don't get these attributes
p2 = Person("Priya")
print(p2.age)      # ❌ AttributeError
                   # p2 doesn't have age
```

3️⃣ CLASS vs INSTANCE ATTRIBUTE — SIDE BY SIDE

python

```
class Car:
```

```
    # —— CLASS ATTRIBUTES ————————————————
    wheels   = 4         # all cars have 4 wheels
    category = "Vehicle"  # all cars are vehicles
    total    = 0         # shared counter

    # —— INSTANCE ATTRIBUTES ————————————————
    def __init__(self, brand, color, speed):
        self.brand = brand  # unique per car
        self.color = color  # unique per car
        self.speed = speed  # unique per car
        Car.total  += 1


car1 = Car("Toyota", "Red",   180)
car2 = Car("Honda",  "Blue",  160)
car3 = Car("BMW",    "Black", 220)

print("—— Class Attributes ——")
print(Car.wheels)     # 4  (same for all)
print(Car.category)   # Vehicle (same for all)
print(Car.total)      # 3  (shared count)

print("—— Instance Attributes ——")
print(car1.brand)     # Toyota
print(car2.brand)     # Honda
print(car3.brand)     # BMW

print(car1.color)     # Red
print(car2.color)     # Blue
print(car3.color)     # Black
```

4️⃣ METHOD ATTRIBUTES

python

```
class Calculator:


    result_history = []   # class attribute


    def __init__(self, name):
        self.name = name  # instance attribute


    # ── INSTANCE METHOD ──────────────────────
    # → Works with instance (self)
    # → Can access instance + class attributes
    def add(self, a, b):
        result = a + b
        self.last_result = result        # instance attr
        Calculator.result_history.append(result)  # class attr
        return result


    # ── CLASS METHOD ────────────────────────
    # → Works with class (cls)
    # → Can only access class attributes
    # → Cannot access instance attributes
    @classmethod
    def get_history(cls):
        return cls.result_history


    @classmethod
    def create_scientific(cls):
        return cls("Scientific")   # creates new instance
```

```
    # —— STATIC METHOD ————————————————
    # → No access to class or instance
    # → Independent utility function
    @staticmethod
    def is_positive(number):
        return number > 0

    @staticmethod
    def multiply(a, b):
        return a * b


# Using all method types
calc = Calculator("Basic")

# Instance method
print(calc.add(10, 20))          # 30
print(calc.add(5,  15))          # 20
print(calc.last_result)          # 20

# Class method
print(Calculator.get_history())     # [30, 20]
sc = Calculator.create_scientific()
print(sc.name)                   # Scientific

# Static method
print(Calculator.is_positive(5))    # True
print(Calculator.multiply(4, 5))    # 20
print(calc.multiply(3, 3))         # 9 (can call via instance too)
```

5️⃣ SPECIAL / MAGIC ATTRIBUTES

python

```python
class Dog:
    """This is a Dog class"""    # docstring

    species = "Canis familiaris"

    def __init__(self, name, age):
        self.name = name
        self.age  = age

    def bark(self):
        return "Woof!"



d = Dog("Tommy", 3)

# ── BUILT-IN MAGIC ATTRIBUTES ──────────────

print(d.__class__)
# <class '__main__.Dog'>

print(d.__dict__)
# {'name': 'Tommy', 'age': 3}
# Shows all instance attributes as dict

print(Dog.__dict__)
# All class attributes + methods as dict

print(Dog.__name__)
# Dog → name of the class
```

```
print(Dog.__doc__)
# "This is a Dog class" → docstring

print(Dog.__module__)
# __main__ → where class is defined

print(Dog.__bases__)
# (<class 'object'>,) → parent classes

print(d.__class__.__name__)
# Dog → class name from instance
```

6️⃣ PRIVATE & PROTECTED ATTRIBUTES

python

```
class BankAccount:

    bank_name  = "SBI"          # public class attribute

    def __init__(self, holder, balance, pin):
        self.holder    = holder   # public instance attr
        self._balance  = balance  # protected instance attr
        self.__pin     = pin      # private instance attr

    # Getter for protected
    def get_balance(self):
        return self._balance

    # Getter for private
    def verify_pin(self, pin):
        return self.__pin == pin
```

```
    # Setter for protected
    def set_balance(self, amount):
        if amount >= 0:
            self._balance = amount

    def __str__(self):
        return f"Account: {self.holder}"


acc = BankAccount("Rahul", 50000, 1234)

# Public → freely accessible
print(acc.holder)         # ✅ Rahul
print(acc.bank_name)      # ✅ SBI

# Protected → accessible but discouraged
print(acc._balance)       # ⚠️ 50000 (bad practice)
print(acc.get_balance())  # ✅ 50000 (correct way)

# Private → not directly accessible
print(acc.__pin)          # ❌ AttributeError
print(acc.verify_pin(1234)) # ✅ True (correct way)

# Name mangling — Python's internal name
print(acc._BankAccount__pin) # ✅ 1234 (via mangling)

# __dict__ shows all instance attributes
print(acc.__dict__)
# {'holder': 'Rahul',
#  '_balance': 50000,
```

```
# '_BankAccount__pin': 1234}
```

7️⃣ ATTRIBUTE METHODS — getattr, setattr, hasattr, delattr

python

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age  = age


p = Person("Rahul", 25)

# ── getattr() ──────────────────────────────
# Get attribute value safely
print(getattr(p, "name"))         # Rahul
print(getattr(p, "age"))          # 25
print(getattr(p, "city", "N/A"))    # N/A (default if not found)
# print(p.city)                   # ❌ AttributeError

# ── setattr() ──────────────────────────────
# Set attribute value dynamically
setattr(p, "city", "Bangalore")
setattr(p, "age", 26)
print(p.city)    # Bangalore
print(p.age)     # 26

# ── hasattr() ──────────────────────────────
# Check if attribute exists
print(hasattr(p, "name"))    # True
print(hasattr(p, "salary"))  # False
```

```
# ── delattr() ──────────────────────────────
# Delete an attribute
delattr(p, "city")
print(hasattr(p, "city"))   # False
# print(p.city)           # ❌ AttributeError now
```

8️⃣ REAL WORLD EXAMPLE — Selenium POM

```python
from selenium import webdriver
from selenium.webdriver.common.by import By

class LoginPage:

    # ── CLASS ATTRIBUTES (Locators) ──────────
    URL      = "https://example.com/login"
    TITLE    = "Login Page"

    # ── INSTANCE ATTRIBUTES ──────────────────
    def __init__(self, driver):
        self.driver   = driver         # browser instance
        self.username = (By.ID, "user") # locator
        self.password = (By.ID, "pass") # locator
        self.btn      = (By.ID, "btn")  # locator
        self._timeout = 10             # protected
        self.__secret = "test123"      # private

    # ── INSTANCE METHODS ─────────────────────
    def open(self):
        self.driver.get(self.URL)

    def login(self, user, pwd):
```

```
        self.driver.find_element(*self.username).send_keys(user)
        self.driver.find_element(*self.password).send_keys(pwd)
        self.driver.find_element(*self.btn).click()

    # ── CLASS METHOD ──────────────────────────
    @classmethod
    def get_url(cls):
        return cls.URL

    # ── STATIC METHOD ─────────────────────────
    @staticmethod
    def is_valid_email(email):
        return "@" in email and "." in email


# Usage in test
driver     = webdriver.Chrome()
login_page = LoginPage(driver)    # instantiation

print(LoginPage.URL)            # class attribute
print(login_page.username)       # instance attribute
print(LoginPage.get_url())       # class method
print(LoginPage.is_valid_email("test@gmail.com"))  # static
```

📊 COMPLETE SUMMARY TABLE

Type	Where Defined	Access Via	Shared	Example
Class Attribute	Class body	Class.attr or obj.attr	✅ All objects	wheels = 4
Instance Attribute	__init__	self.attr or obj.attr	❌ Own copy	self.name
Protected Attribute	__init__	obj._attr (discouraged)	❌ Own copy	self._balance

Type	Where Defined	Access Via	Shared	Example
Private Attribute	__init__	Only inside class	❌ Own copy	self.__pin
Class Method	Class body	Class.method()	✅ All objects	@classmethod
Static Method	Class body	Class.method()	✅ All objects	@staticmethod
Magic Attribute	Built-in	obj.__attr__	Varies	__dict__

💬 **INTERVIEW ANSWER**

Q: What is the difference between
attributes and variables in Python OOP?

A: Variables are general storage containers
in Python. When variables are defined
inside a class, they are called attributes.

There are two main types of attributes:

1. Class Attributes
   → Defined in class body outside methods
   → Shared by ALL instances
   → Accessed via ClassName.attribute
   → Example: wheels = 4 in Car class

2. Instance Attributes
   → Defined inside __init__ using self
   → Unique to EACH instance/object
   → Accessed via self.attribute
   → Example: self.name = name

Key difference:
→ Class attributes are SHARED

→ Instance attributes are INDIVIDUAL

Additionally attributes can be:

→ Public (name)

→ Protected (_name)

→ Private (__name)

→ Magic (__name__)

💡 **One line to remember: "Variables become Attributes when they live inside a Class!"**